

RAG-Based Customer Support Assistant

High-Level Design (HLD)

Name:T.Bhargavi

Intern ID:IN226082802

1. System Overview

Problem Definition

Customer support systems often fail to provide accurate and context-aware responses when relying on static FAQs or traditional search systems. This leads to poor user experience and increased dependency on human agents.

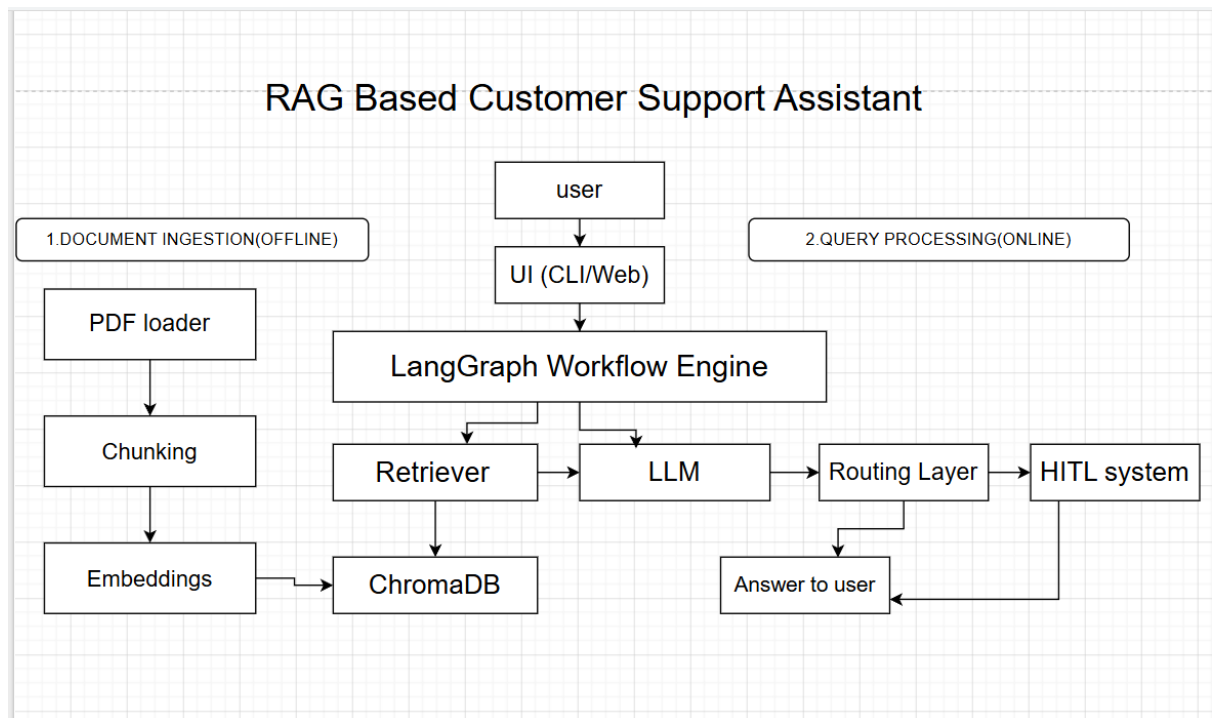
Solution

The proposed system is a Retrieval-Augmented Generation (RAG)-based Customer Support Assistant that leverages a PDF knowledge base to generate accurate, context-aware responses to user queries.

Scope

- Supports a single PDF knowledge base
- Handles customer queries via CLI/Web interface
- Retrieves relevant information using embeddings
- Generates responses using an LLM
- Supports Human-in-the-Loop (HITL) escalation for complex queries

2. Architecture Diagram



2. Architecture Diagram

The above diagram represents the high-level architecture of the RAG-based customer support assistant. It consists of two main flows:

1. Document Ingestion (Offline)
2. Query Processing (Online)

The system uses LangGraph for workflow orchestration and ChromaDB as the vector database for storing embeddings.

3. Component Description

- **User:** Initiates queries to the system.
- **UI (CLI/Web):** Interface through which users interact with the system.
- **PDF Loader:** Loads the knowledge base document.

- **Chunking Module:** Splits the document into smaller chunks for processing.
- **Embedding Model:** Converts text chunks into vector representations.
- **ChromaDB (Vector Store):** Stores embeddings for efficient similarity search.
- **Retriever:** Fetches relevant chunks based on user query.
- **LLM (Large Language Model):** Generates responses using retrieved context.
- **LangGraph Workflow Engine:** Manages the flow of data between components.
- **Routing Layer:** Determines whether to respond or escalate.
- **HITL System:** Handles queries requiring human intervention.

4. Data Flow

Document Ingestion Flow:

PDF → Chunking → Embeddings → ChromaDB

Query Processing Flow:

User → UI → LangGraph → Retriever → ChromaDB → LLM → Routing Layer

- If confidence is high → Answer returned to user
- If confidence is low → Escalated to HITL system

5. Technology Choices

- **ChromaDB:** Lightweight and efficient vector database for storing embeddings.

- **LangGraph:** Enables graph-based workflow orchestration and conditional routing.
- **LLM (OpenAI / HuggingFace):** Used for generating contextual responses.
- **LangChain:** Facilitates integration of LLMs, retrievers, and vector stores.

6. Scalability Considerations

- Large documents can be handled using efficient chunking strategies.
- Query load can be managed using caching and asynchronous processing.
- Vector database can be scaled using cloud-based solutions like Pinecone.
- Latency can be reduced by optimizing retrieval and limiting chunk size.